

Graduated, Two-Mode Risk Scoring for AI Agent Access to Model Context Protocol Servers

Tomer Ovadya

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev
Beer-Sheva, Israel
ovadyat@post.bgu.ac.il

Asaf Shabtai

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev
Beer-Sheva, Israel

Abstract—The Model Context Protocol (MCP) lets AI agents invoke real-world tools—file operations, database queries, and API calls—through a uniform interface, spawning an ecosystem of more than 67,000 servers. To the server that owns those resources, every incoming agent request is a potential threat: one malicious or misguided call can exfiltrate secrets, corrupt records, or trigger cascading side effects. Existing MCP defenses, however, ultimately collapse to a binary allow/deny verdict, discarding any internal severity signal before it reaches the operator and ignoring the spectrum of risk between a safe read and a destructive write. We present *McpRisk*, a defense-oriented framework that produces a graduated risk score for MCP tool invocations in two modes: a static score computed at design time from a server’s tool registry and asset inventory alone, and a dynamic score computed at request time from concrete arguments, session context, and call frequency. Scores follow an auditable $\text{Impact} \times \text{Likelihood} \times \text{Irreversibility}$ model and are produced by a multi-stage LLM pipeline with an independent reviewer stage and a deterministic offline fallback. We evaluate against external, third-party ground truth: three public MCP attack benchmarks (MCPSecBench, MCP-SafetyBench, MSB; 150 labeled calls under the same server-as-victim threat model), graded as an attack-detection task. The contrast is sharp: capability-only risk frameworks (CVSS, AIVSS) sit at chance (AUC 0.52) because the attack lives in the call’s arguments, not the tool, whereas request-aware scoring—an LLM judge over (tool, args, server), the form our dynamic mode takes—is the strongest detector (AUC 0.68, 42% recall at a 10% false-positive budget). A positioning matrix shows *McpRisk* is the only work that scores graduated, per-(tool, asset) MCP risk in both a design-time and a request-time mode under this threat model and validates it on external benchmarks. We release the framework, the baselines, and the evaluation pipeline.

Index Terms—Model Context Protocol, AI agents, risk scoring, LLM security, access control, tool invocation.

I. Introduction

The Model Context Protocol (MCP), introduced by Anthropic in late 2024, standardizes how AI agents connect to external tool servers through a host–client–server architecture, exposing tools, resources, prompts,

and sampling as first-class primitives. Adoption has been explosive: an empirical census counts more than 67,000 servers across six public registries, exposing over 44,000 tools [1]. Each tool can read a file, mutate a database, send a message, or call a remote API—real, often irreversible actions on resources the server owns.

This growth has outpaced security. Servers are contributed by untrusted third parties with no mandatory review, and the protocol assumes that tool metadata is trustworthy [2]. A growing literature documents the resulting attack surface: tool poisoning that steers agent behavior through crafted metadata [3], [4], parasitic toolchains whose danger emerges only from tool combinations [5], protocol-level exploits [6], and covert exfiltration through ostensibly benign logging tools [7].

Our vantage point is the server as the protected asset. Whether a request originates from a benign-but-clumsy agent, a prompt-injected one, or an outright malicious client, the server must decide—before execution—whether to let the call through. The defenses proposed so far ultimately collapse this decision to a binary verdict—trusted/untrusted [8], safe/unsafe [9]–[11], or allowed/blocked [12], [13]. Even systems that reason internally over a continuous trust or confidence score [8] expose only the thresholded outcome, so an operator cannot set proportionate responses. A binary gate is badly matched to reality: reading a public README and deleting a directory of private keys are not the same event, yet a single threshold must either over-block routine work or under-protect crown-jewel assets.

We argue that MCP needs graduated, context-aware risk quantification, in the spirit of CVSS for software vulnerabilities. Recent results show that LLMs can predict CVSS-style severity from natural-language descriptions with roughly 79% accuracy [14], and that effective agent risk signals must be dynamic, contextual, and compos-

able [15]. Crucially, prior graduated scoring is performed once per static vulnerability (CVSS) or at the level of system governance (TRiSM), not per request for individual MCP tool calls—which is the granularity a gate needs. Contributions. This paper makes the following contributions:

- A defense-oriented threat model that fixes the MCP server as the victim and the agent as the threat, together with an auditable $\text{Impact} \times \text{Likelihood} \times \text{Irreversibility}$ risk model adapted from vulnerability scoring, including an explicit likelihood aggregator and band map (Sections II–IV).
- A two-mode design—a static design-time score over a server’s tools and assets, and a dynamic request-time score that can escalate a call above its static ceiling (Section V).
- An evaluation on external, third-party MCP attack benchmarks framed as detection, showing capability-only frameworks (CVSS, AIVSS) are at chance while request-aware scoring detects, together with a positioning matrix against competing risk-scoring works (Section VI).

II. Threat Model

Throughout this work the MCP server is the protected asset (the victim). The attacker is a malicious or compromised AI agent—or the human behind it—that sends harmful requests through the MCP protocol to damage, exfiltrate from, or disrupt the server and the resources it manages. All attacks flow in the client→server direction. Concretely, the defender is a gateway sitting in front of the server (Fig. 1) that observes each tool call’s name, target asset, and arguments, plus whatever session history is available, and must issue a verdict before the call executes. The defender does not rely on anomaly detection or known-bad signatures; it instead defines, explicitly and auditably, what makes an interaction risky.

We deliberately exclude the inverse scenario in which a malicious server attacks the agent (e.g., tool poisoning that tricks the agent into harmful behavior); that is a complementary but distinct direction [3], [16]. Where poisoning is relevant here, it is only insofar as a poisoned agent subsequently becomes a threat to the server.

III. Background and Related Work

We organize prior work into four threads that together motivate graduated scoring.

Ecosystem and attack surface. Hou et al. [2] give the first systematic analysis of MCP, cataloging four attacker types and 16 threat scenarios over the server lifecycle. Li and Gao [1] quantify scale—67,057 servers, 44,499 tools—and find leaked credentials, server-hijacking flaws, and 408 groups of confusingly similar tool names enabling squatting; none of the four major hosts they audit implements graduated risk assessment. Threat taxonomies enumerate the space the defender must cover: Ferrag et

al. [17] span 30+ techniques across input manipulation, model exploitation, system compromise, and protocol abuse; Zhao et al. [18] identify 12 attack categories over six server components; and Maloyan and Namiot [6] show specification-level weaknesses that amplify prompt-injection success by 23–41%.

Tool poisoning and parasitic attacks. MCPTox [3] reports 72.8% attack success for explicit tool poisoning over 45 servers and 353 tools, while MCP-ITP [4] achieves 84.2% success with implicit poisoning that evades detection at a 0.3% rate by causing misuse of other tools. Parasitic toolchains [5] reach roughly 90% success from dangerous tool combinations, and Log-To-Leak [7] weaponizes benign logging tools for covert exfiltration—so output quality cannot serve as a safety signal. MindGuard [16] inspects the agent’s internal decision process to detect metadata-driven misuse. These results imply a risk model must weigh tool capability, asset sensitivity, and data-flow context—not just the surface text of a single call.

Defenses and access control. MCPShield [8] builds an adaptive “security cognition” by probing servers and reasoning over execution traces, internally computing a continuous deny signal that it then thresholds to a trusted/untrusted decision; it defends the agent from untrusted servers (the inverse of our direction) and reports defense rates across several attack benchmarks. MCP-Guard [9] cascades fast pattern matching, a fine-tuned neural classifier, and LLM arbitration, reaching an F1 of 95.4% on its full pipeline, yet still emits a binary safe/unsafe verdict. ToolSafe [10] and GuardAgent [11] likewise yield categorical decisions. On the access-control side, Progent [12] cuts attack success from about 39.9% to roughly 1–2% (and to near zero with hand-written policies) using programmable privilege-control policies; AgentBound [13] makes access control a first-class primitive with OS-level enforcement; and Wu et al. [19] learn minimal-privilege data-access policies. All enforce binary boundaries or expose only a thresholded verdict, and none surfaces the degree of risk of a permitted action.

Risk quantification. Jafarikhah et al. [14] show LLMs can predict CVSS base metrics from descriptions ($\approx 79\%$ accuracy), establishing text-based scoring as feasible—but once per static vulnerability. Raza et al. [15] survey 180 papers on Trust, Risk, and Security Management for agentic AI and themselves propose composite risk scores, but at the level of system governance rather than per-invocation MCP tool calls. No prior system produces a graduated, per-(tool, asset), request-instantiated risk band for MCP gating—the gap McpRisk fills. We position our work as a complement that supplies the risk evidence the binary defenses above currently discard.

IV. Risk Model

McpRisk scores a tool interaction as a composition of three auditable factors:

$$\text{Risk} = \underbrace{(\text{Sensitivity} \times \text{Blast})}_{\text{Impact}} \times \text{Likelihood} \times \text{Irreversibility}. \quad (1)$$

Impact captures how bad a successful action is, along two independent scales. Sensitivity $s \in [1, 5]$ reflects the confidentiality/value of the asset touched, from public config (1) to regulated data, PII, and secrets (5). Blast radius $b \in [0, 4]$ reflects how far one action propagates, from no reach (0) through a single record (1) and a departmental table (3) to systemic, all-users scope (4). Irreversibility is instantiated by the per-tool impact level $i \in \{1, 2, 3\}$: a read-only action ($i=1$, recoverable), a recoverable write ($i=2$), or a destructive/irreversible action ($i=3$).

Likelihood $L \in [0, 1]$ estimates how anomalous, and thus how plausibly malicious, the action is at runtime, from three sub-factors: baseline deviation $d \in [0, 1]$ (how unusual the action is for this principal’s history), access anomaly $a \in [0, 0.9]$ (timing/volume outlier), and exposure $e \in [0, 0.9]$ (how much untrusted external content has entered the context window, the prompt-injection surface). We aggregate them with a bounded, monotone noisy-OR so that no single context factor saturates likelihood and history (d) remains the only factor that can reach 1:

$$L = 1 - (1 - d)(1 - a)(1 - e), \quad a, e \leq 0.9. \quad (2)$$

Range and bands. With $s \in [1, 5]$, $b \in [0, 4]$, $L \in [0, 1]$, $i \in \{1, 2, 3\}$, the raw score lies in $[0, 60]$. Banding the raw number alone makes $\sim 10\%$ of cells critical, which would halt legitimate work, so McpRisk maps scores to four operational bands with an explicit, auditable band map B rather than a single cutoff. Writing $r = s \cdot b \cdot L \cdot i$:

$$B = \begin{cases} \text{critical} & i=3 \wedge s=5 \wedge b \geq 3 \\ \text{high} & (i=3 \wedge s \geq 4) \vee r \geq 24 \vee (i=1 \wedge s \geq 4 \wedge b \geq 3) \\ \text{medium} & r \geq 8 \vee (i=1 \wedge s=5) \\ \text{low} & \text{otherwise.} \end{cases}$$

The map reserves critical for irreversible destruction of a crown-jewel asset (sensitivity 5) at departmental-or-wider reach, keeping it rare ($\sim 1\text{--}2\%$ of cells). A confidentiality floor keeps reads of secrets at medium (narrow) or high (broad) and never critical, since a read changes no state but still leaks. Bands drive proportionate responses: auto-allow, log-and-monitor, escalate-to-human, and auto-deny. Two modes. The same model is evaluated at two times. In the static mode (design time), likelihood is pinned to its upper bound $L=1$ (Eq. 2 with $d=1$): the score is a worst-case ceiling derived from tool and asset properties alone, answering “which (tool, asset) pairs on this server are inherently dangerous?” In the dynamic mode (request time), the concrete arguments, session context, and frequency instantiate L and can escalate a call above its static cell.

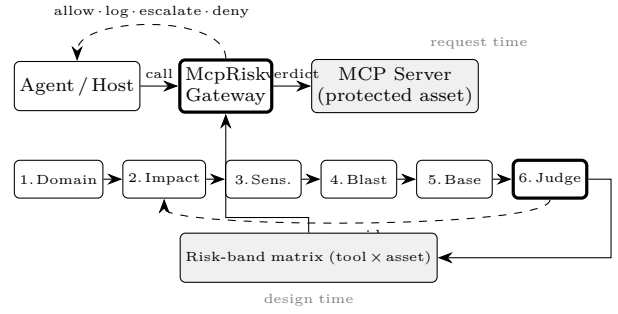


Fig. 1. McpRisk architecture. At design time (bottom) a six-stage pipeline derives a risk-band matrix from the server’s tools/list registry and on-disk store, with an independent judge (stage 6) overriding earlier stages. At request time (top) the gateway consults that matrix—and the dynamic scorer—to gate each call before it reaches the server. Bold boxes are McpRisk-owned components.

Worked example. Consider `edit_file` on `private_key.pem`: $s=5$, $b=3$, $i=3$, $L=1$, so $r=45$ and $B=\text{critical}$. A `read_file` on the same asset has $i=1$, so $r=15$ but the broad-read rule yields high. At request time, a recursive/bulk read raises the parameter band, which escalates the call’s final band upward—capturing mass exfiltration that the inherent read cell alone would miss.

V. Design and Implementation

McpRisk is implemented in Python (3.12+) and uses a locally hosted open-weight LLM (Qwen2.5 served through Ollama) for all judgment stages, so that no tool metadata or asset inventory leaves the deployment.

A. Static Scanner

The static scanner runs before a server is live and never connects to a running server. It assembles a server description from two static sources: the server’s own saved tools/list registry, and the on-disk store the server fronts (a filesystem tree, a SQL schema, or seeded channels). It then derives every risk primitive through the six-stage pipeline of Fig. 1: (1) domain inference, so later stages interpret assets in context; (2) tool impact (i); (3) asset sensitivity (s); (4) blast radius (b); (5) per-application baselines; and (6) a judge cross-check in which an independent reviewer re-derives every primitive from the same domain profile and overrides the proposer on disagreement. The pipeline then computes the cell matrix $\text{cells} = s \times b \times L \times i$ (with $L=1$ at design time) and applies the band map B .

Two design choices make the output trustworthy as a security artifact. First, nothing is hardcoded and the reference tables are never read: if the model is unreachable, a stage either falls back to documented deterministic heuristics (and flags the table needs `human_review`) or raises rather than guess. Second, decoding is greedy (temperature 0); the offline heuristic path is bit-exact, and on our fixed model and hardware repeated LLM runs produced identical bands, which is the property an

enforcement path requires. (We do not claim byte-identity across arbitrary GPU/driver/quantization changes.) Every judge override is logged in a `crosscheck_summary`, giving a reviewer a complete decision trail.

B. Dynamic Call Scorer

At request time, the call scorer resolves an observed call to a scanned (tool, asset) cell and reports that cell’s score and band. Crucially, it never fabricates a score: a call to a tool absent from the scan is reported invalid (a misconfiguration signal), and a known tool whose target is not a scanned cell (e.g., a directory enumeration with no single-file asset) is reported unresolved. When a per-tool parameter rubric is supplied, the scorer computes the input-parameter risk and escalates the inherent cell band into a final band via a monotone max (it never lowers risk), so a large or bulk argument can raise a call above its static baseline. This realizes the dynamic half of Eq. 1 while preserving the honesty property that only verified cells produce numeric scores.

VI. Evaluation

A. Setup

We built a reference corpus of seven realistic MCP servers spanning three kinds: four filesystem servers (a generic corporate store and three domain stores—a law firm, a media studio, and a medical clinic), two SQL servers, and one communication (Slack) server, comprising 74 tools and a heterogeneous asset inventory from public documents to private keys, API keys, invoices, and patient records. Each server has a per-server, judge-reviewed design-time reference table held out as ground truth; across the corpus these contain 5 critical cells and 52 recorded judge overrides. We evaluate the static scanner on the five servers we have scanned to date (the four filesystem servers and one SQL server, 61 tools); the remaining two servers (a second SQL server and Slack) have reference tables but were not yet scanned and so do not contribute to scanner agreement. These design-time tables serve only as a sanity check on the static primitives (Sec. VI-B); they are LLM-reviewed and thus not fully independent, so we do not lean on them for the headline result.

The decisive evaluation is external. We use three public, third-party MCP attack benchmarks—MCPSecBench, MCP-SafetyBench, and MSB—comprising 150 labeled tool calls (55 ATTACK with a 17-type attack taxonomy, 59 benign VALID, and misconfiguration calls). They were authored by other groups, are entirely independent of our framework, and share its threat model: the MCP server is the protected asset and calls flow client→server. We treat each scorer’s per-call risk as a detector of ATTACK (Sec. VI-C) and additionally position `McpRisk` against competing risk-scoring works (Sec. VI-D).

TABLE I

Per-server static-scanner agreement on tool impact (read-only / recoverable / destructive). Fractions are matching tools / total. Two further reference servers (a SQL store and Slack) were not yet scanned.

Server	Kind	Tools	Tool-impact
corp_filesystem	filesystem	14	14/14
law_firm_fs	filesystem	14	13/14
media_studio_fs	filesystem	14	14/14
medical_clinic_fs	filesystem	14	13/14
cbg_sqlite	SQL	5	5/5
Total		61	59/61 (97%)

TABLE II

Static-scanner agreement by risk primitive. A cell is one (tool, asset-class) pair. Band/sensitivity agreement is reported on the SQL server whose asset keys align with the reference; tool impact spans all five scanned servers.

Primitive	Scope	Agreement
Tool impact (1–3)	5 servers, 61 tools	59/61 (97%)
Asset sensitivity (1–5)	SQL, 7 classes	5/7 (71%)
Composed band (exact)	SQL, 35 cells	12/35 (34%)

B. Static Sanity Check

Tool impact. Table I shows the scanner reproduces the held-out reference’s tool-impact classification on 59 of 61 tools (97%), and the agreement is uniform across servers (13–14 of 14 on every filesystem server, 5/5 on SQL) rather than carried by one easy case. This is the simplest primitive—whether a tool reads, writes, or destroys—so we treat it as a sanity diagnostic, not the headline result. **Composed band.** The fully composed band against the design-time reference agrees exactly on 34% of the SQL server’s 35 cells (asset sensitivity 71%; Table II). Exact agreement is low because a band is the product of four primitives—a single-level disagreement on any factor shifts the composed band even when each primitive is individually close. We do not lean on this internal reference: the decisive test is detection on external, third-party-labeled benchmarks (Sec. VI-C).

Error analysis. The scanner’s residual errors are systematic and conservative rather than random. It over-weights single-object irreversibility (rating some irreversible writes/moves critical) and under-weights enumeration as an exfiltration primitive (rating broad directory listings low). On SQLite it is conservative on reads of crown-jewel tables (e.g., `api_keys`, `employees`), scoring them high/medium rather than critical; this is a false-negative on the security-critical cells, and is the most important limitation of the static view. These are precisely the cases that argument-level, request-time scoring must address, and the external detection results below (Sec. VI-C) quantify exactly how far a capability-class-level view falls short.

TABLE III

Attack detection on external MCP benchmarks (MCPSecBench, MCP-SafetyBench, MSB; 55 ATTACK vs. 59 VALID). AUC is threshold-free; recall@10%FPR is the catch rate at a tight false-positive budget.

Scorer	AUC	Recall@10%FPR
LLM judge (request-aware)	0.68	42%
Keyword detector (content)	0.67	38%
CVSS v3 (capability-only)	0.52	0%
OWASP AIVSS (capability-only)	0.52	0%
Majority / random (floor)	≈ 0.50	—

C. External-Benchmark Detection: Risk Scorers as Attack Detectors

The decisive test uses external, third-party ground truth. We treat each scorer’s per-call risk as a detector of ATTACK on three public MCP benchmarks (150 labeled calls; 55 ATTACK vs. 59 benign VALID) and report AUC (threshold-free ranking quality) and recall at a 10% false-positive budget—the way the guardrail literature evaluates [9], [11]. We compare capability-only frameworks (CVSS, AIVSS), which score the tool; a content-aware keyword detector that reads the arguments; and a request-aware LLM judge over (tool, args, server)—the form our framework’s dynamic mode takes.

Table III carries the central comparison. The capability-only frameworks are at chance (AUC 0.52, 0% recall at the FPR budget): on these benchmarks the maliciousness lives in the arguments—a path-traversal `../etc/passwd`, an injected `<IMPORTANT>...` string, an exfiltration payload—while CVSS and AIVSS see only the tool, which is identical across a tool’s benign and attack calls. McpRisk’s static band is itself capability-class-level (Table I confirms it captures tool impact), so it shares this blind spot by construction; detecting these attacks requires reading the request. The request-aware scorers do: the LLM judge that instantiates our dynamic mode is the strongest detector (AUC 0.68, 42% recall@10%FPR), edging the keyword detector and far above the capability frameworks, and leads on each of the three benchmarks individually. This is external, third-party evidence for the paper’s thesis: per-call MCP risk needs request-aware scoring, not a richer static table. We also report the honest ceiling—even the best detector reaches only AUC 0.68, because the benchmarks include subtle tool-poisoning and rug-pull payloads with benign-looking arguments; per-call MCP attack detection remains open.

D. Positioning vs. Other Risk-Scoring Work

Table IV places McpRisk against the closest risk-scoring works. No prior approach occupies the same cell: capability scorers (CVSS, AIVSS, the Entra permission-risk model, description-to-CVSS) are per-vulnerability and argument-blind; agent-safety detectors (R-Judge, AgenTRIM, AURA) emit a binary or allow/deny verdict

TABLE IV

Positioning vs. neighbouring risk scorers. Grad.=graduated output (vs. binary); Per-call=per-invocation granularity; Arg=reads the request arguments; Srv=server-as-victim threat model; Ext.GT=evaluated on external labeled data.

Work	Grad.	Per-call	Arg	Srv	Ext. GT
McpRisk (ours)	✓	✓	part	✓	✓
CVSS v3 / AIVSS	✓	—	—	—	✓
Permission-risk (Entra)	✓	—	—	—	✓
AgenTRIM	—	✓	part	part	—
R-Judge	—	✓	✓	part	✓
AURA	✓	part	—	—	—
MCP-RiskCue	✓	part	✓	✓	part

and are evaluated mostly by downstream task success, not graduated risk. McpRisk is the only one that scores graduated, per-(tool, asset) MCP invocation risk in both a design-time and a request-time mode under the server-as-victim threat model and validates it on external attack benchmarks.

E. Threats to Validity

The external benchmarks are modest in size (150 calls across three suites) and skew toward tool-poisoning attacks, so the absolute AUCs are indicative rather than definitive; we report them with a fixed-FPR operating point and a chance floor to keep them honest. The design-time reference tables used for the static sanity check (Tables I–II) were judge-reviewed by the same LLM family the scanner uses, so they establish process—not prior—independence, and the demo servers are author-built. Crucially, the request-aware detector we report is a standalone LLM judge; integrating it as McpRisk’s dynamic stage and evaluating the full two-mode gate end-to-end—together with larger external benchmarks and real registry servers—is the necessary next step.

VII. Discussion and Limitations

McpRisk is positioned as a complement to, not a replacement for, the binary defenses surveyed in Section III. A graduated score is most valuable where those systems are weakest: deciding how to respond once an action is deemed non-trivially risky. By emitting a band rather than a verdict, the framework lets an operator tune the allow/log/escalate/deny thresholds to their risk appetite without retraining a classifier.

Two limitations bound the current design. First, the dynamic score’s richness depends on how much session context the gateway can observe at runtime; with little visibility into agent history, the likelihood term (Eq. 2) degrades toward its static upper bound. Second, an LLM-based scorer inherits the biases and failure modes of its base model; the independent judge stage and the deterministic heuristic fallback mitigate but do not eliminate this. We treat both as explicit, audited properties rather than hidden assumptions, and the error analysis above

shows the residual failure mode (conservative scoring of crown-jewel reads) is the one the dynamic mode is built to correct.

VIII. Conclusion

We presented McpRisk, a defense-oriented framework that treats the MCP server as the protected asset and produces a graduated, auditable risk score for agent tool invocations in two modes—static at design time and dynamic at request time—following a transparent $\text{Impact} \times \text{Likelihood} \times \text{Irreversibility}$ model with an explicit likelihood aggregator and band map. On external, third-party MCP attack benchmarks framed as detection, capability-only risk frameworks (CVSS, AIVSS) sit at chance because the attack lives in the call’s arguments, while request-aware scoring—an LLM judge over (tool, args, server), the form our dynamic mode takes—is the strongest detector (AUC 0.68 vs. 0.52), and a positioning matrix shows no prior work scores graduated, per-(tool, asset) MCP risk in both modes under the server-as-victim model. By replacing a single allow/deny threshold with a calibrated, operator-tunable risk band, McpRisk gives MCP servers an auditable basis for proportionate, context-aware enforcement; integrating the request-aware detector as the dynamic stage and evaluating the full two-mode gate on larger external benchmarks are the immediate next steps.

References

- [1] X. Li and X. Gao, “Toward understanding security issues in the model context protocol ecosystem,” arXiv preprint arXiv:2510.16558, 2025.
- [2] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model context protocol (MCP): Landscape, security threats, and future research directions,” arXiv preprint arXiv:2503.23278, 2025.
- [3] Z. Wang, Y. Gao, Y. Wang et al., “MCPTox: A benchmark for tool poisoning attack on real-world MCP servers,” arXiv preprint arXiv:2508.14925, 2025.
- [4] R. Li, Z. Wang, Y. Yao, and X.-Y. Li, “MCP-ITP: An automated framework for implicit tool poisoning in MCP,” arXiv preprint arXiv:2601.07395, 2026.
- [5] S. Zhao, Q. Hou, Z. Zhan et al., “Mind your server: A systematic study of parasitic toolchain attacks on the MCP ecosystem,” arXiv preprint arXiv:2509.06572, 2025.
- [6] N. Maloyan and D. Namiot, “Breaking the protocol: Security analysis of the MCP specification and prompt injection vulnerabilities,” arXiv preprint arXiv:2601.17549, 2026.
- [7] S. Hu et al., “Log-to-leak: Prompt injection attacks on tool-using LLM agents via model context protocol,” Open-Review, 2026, available at <https://openreview.net/forum?id=UVgbFuXPao>.
- [8] Z. Zhou, Y. Zhang, H. Cai et al., “MCPShield: A security cognition layer for adaptive trust calibration in MCP agents,” arXiv preprint arXiv:2602.14281, 2026.
- [9] W. Xing, Z. Qi, Y. Qin et al., “MCP-Guard: A multi-stage defense-in-depth framework for securing MCP in agentic AI,” arXiv preprint arXiv:2508.10991, 2026.
- [10] Y. Mou, Z. Xue, L. Li et al., “ToolSafe: Enhancing tool invocation safety via proactive step-level guardrail and feedback,” arXiv preprint arXiv:2601.10156, 2026.
- [11] Z. Xiang, L. Zheng, Y. Li et al., “GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning,” arXiv preprint arXiv:2406.09187, 2025.
- [12] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, “Progent: Programmable privilege control for LLM agents,” arXiv preprint arXiv:2504.11703, 2025.
- [13] C. Buhler, M. Biagiola, L. Di Grazia, and G. Salvaneschi, “Securing AI agent execution,” arXiv preprint arXiv:2510.21236, 2025.
- [14] S. Jafarikhah, D. Thompson, E. Deans, H. Siadati, and Y. Liu, “From description to score: Can LLMs quantify vulnerabilities?” in Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing (SAC ’26), 2026.
- [15] S. Raza, R. Sapkota, M. Karkee, and C. Emmanouilidis, “TRiSM for agentic AI: A review of trust, risk, and security management,” arXiv preprint arXiv:2506.04133, 2025.
- [16] Z. Wang, H. Du et al., “MindGuard: Intrinsic decision inspection for securing LLM agents against metadata poisoning,” arXiv preprint arXiv:2508.20412, 2026.
- [17] M. A. Ferrag, N. Tihanyi et al., “From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows,” arXiv preprint arXiv:2506.23260, 2025.
- [18] W. Zhao, J. Liu, B. Ruan et al., “When MCP servers attack: Taxonomy, feasibility, and mitigation,” arXiv preprint arXiv:2509.24272, 2025.
- [19] Y. Wu, K. Yang, F. Roesner et al., “Towards automating data access permissions in AI agents,” in Proceedings of the IEEE Symposium on Security and Privacy (S&P), 2026.